

Escuela Técnica Superior de Ingeniería

Prácticas de Programación Concurrente y Distribuida

3º Curso de Grado en Ingeniería Informática

PRÁCTICA 3

Sincronización y Creación de *Frames* en JAVA

El objetivo de la práctica es conocer la forma de sincronizar métodos de objetos concurrentes y familiarizarse con la creación de *Frames* y la representación gráfica a través de la clase `Canvas`.

Se tomará como base la cola implementada tras la práctica 2 (`ColaLenta`).

1. Se debe modificar la clase `ColaLenta` para evitar los problemas derivados del acceso concurrente de los hilos, observados en la práctica 2. Para ello se sincronizarán los métodos `acola` y `desacola`.

2. Se debe crear un nuevo *Frame* que lance un hilo `Productor` y otro hilo `Consumidor` y represente gráficamente la evolución de los elementos en la cola, según son insertados o extraídos. Para tal fin, se creará la estructura de clases que representa el diagrama de clases de la figura 1.

La función `main` de la clase `ColaFrame` realizarán las siguientes acciones:

- Crear los objetos `ColaFrame`, `ColaLenta`, `CanvasCola`, y los objetos `Productor` y `Consumidor`.

- Definirá el tamaño y colores del *Frame* y añadirá el *Canvas* al mismo.

- Lanzará los hilos `Productor` y `Consumidor`.

- Esperará a que los hilos `Productor` y `Consumidor` finalicen, esperará tres segundos y finalizará la ejecución.

Los métodos de la clase `CanvasCola` realizarán las siguientes acciones:

- `update`: Este método sobrecarga el método `update` de la clase `Canvas`. Es invocado cada vez que se invoca al método `repaint` de la clase `Canvas`. El método `update` de `Canvas` borra el *Canvas*, lo repinta con el color del fondo e invoca al método `paint`. Para evitar problemas de rendimiento en la visualización, se debe sobrecargar este método en `CanvasCola` para que invoque al método `paint`. Con esto se consigue que no se borre el *canvas* cada vez que se invoca a `repaint`.

- `paint`: Es el encargado de dibujar en el *Canvas*. Lo invoca automáticamente `update` tras una llamada a `repaint`. Para mejorar la representación se usa una imagen a modo de *buffer* sobre la que se dibuja, en lugar de hacerlo sobre el parámetro `g` de tipo `Graphic`. Una vez compuesta la imagen en el *buffer* se dibuja en `g`.

- `avisa`: será invocado cada vez que se desee mostrar el mensaje de "*COLA LLENA*" o "*COLA VACIA*". Tomará como parámetro el mensaje a mostrar. Una vez modificada la información, invocará a `repaint` para actualizar la información del *Canvas*.

- `representa`: será invocado cada vez que haya cambios en la cola que deban ser representados en pantalla. Toda la información sobre el estado de la cola será recogida en los parámetros pasados. Una vez modificada la información, invocará a `repaint` para actualizar la información del *Canvas*.

La clase `ColaLenta` deberá ser modificada para representar la información cada vez que cambia. Para ello, deberá tener una referencia a la instancia de `CanvasCola` creada en `ColaFrame` e invocar a los métodos `avisa` y `representa` según se *acolen* o *desacolen* elementos de la cola.

Figura 1: Diagrama de Clases de la práctica 3

